T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 23, 2026	**DATE SUBMITTED:**	July 28, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 2.1 **Linked Lists**

1. PROCEDURE OUTPUT

Screenshot

Header File Code:

singly_LL.h

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

//TEMPLATING
template <typename T>
class SinglyList{
    public:
        T data;
        SinglyList<T> *next = nullptr;

};
```

Main Code:

mainSingly.cpp > main()

```
#include <iostream>
#include "singly_LL.h"

int main(){

    // Creating an Empty List
    SinglyList<char>* head = new SinglyList<char>;
    SinglyList<char>* second = new SinglyList<char>;
    SinglyList<char>* third = new SinglyList<char>;
    SinglyList<char>* fourth = new SinglyList<char>;
    SinglyList<char>* fifth = new SinglyList<char>;

    // Assign the Data
    head->data = 'N';
    head->next = second;

    second->data = 'I';
    second->next = third;

    third->data = 'N';
    third->next = fourth;

    fourth->data = 'Y';
    fourth->next = fifth;

    fifth->data = 'A';
    fifth->next = NULL;

    std::cout << head->data << second->data << std::endl;
    std::cout << head->next << second->next->data << std::endl;
    listTraversal(head);

```

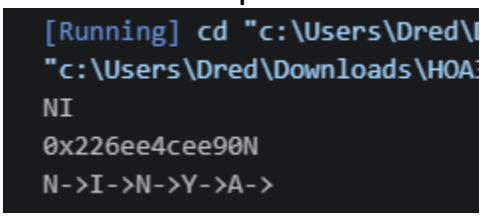
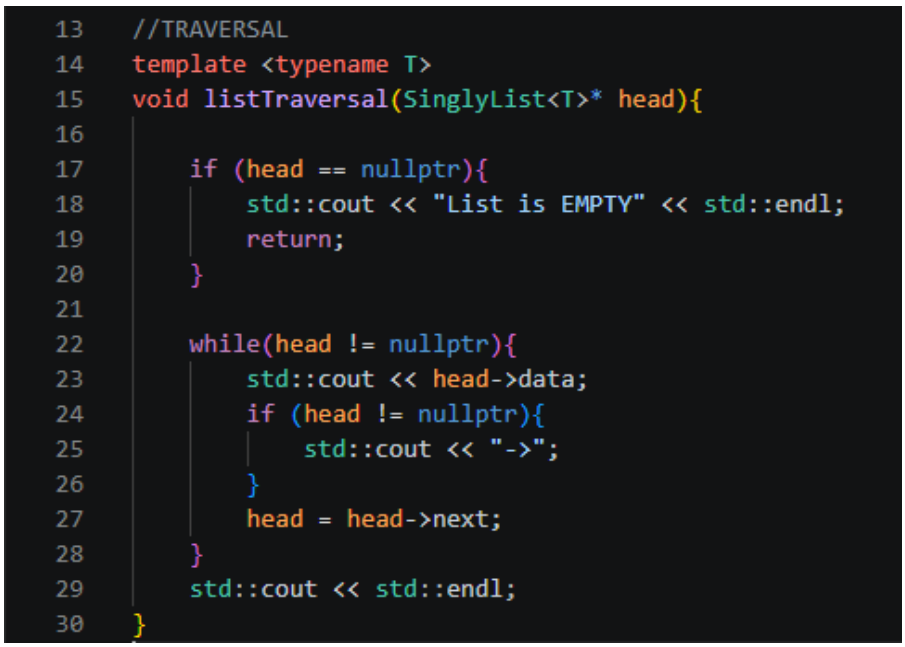
	Output: 
Discussion	In this code, nodes are created manually one by one (head, second, third, etc.), and each node is assigned a character as its data. The nodes are then linked together using the next pointer, forming a chain that spells out the sequence N -> I -> N -> Y -> A. This demonstrates the basic structure of a singly linked list, where each node points to the next until the last node points to NULL.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

```
13  //TRAVERSAL
14  template <typename T>
15  void listTraversal(SinglyList<T>* head){
16
17      if (head == nullptr){
18          std::cout << "List is EMPTY" << std::endl;
19          return;
20      }
21
22      while(head != nullptr){
23          std::cout << head->data;
24          if (head != nullptr){
25              std::cout << "->";
26          }
27          head = head->next;
28      }
29      std::cout << std::endl;
30  }
```

Insertion at Head

```
31
32  //INSERT HEAD
33  template <typename T>
34  void sllInsertHead(T newData, SinglyList<T>** currentHead){
35      // 1. Allocate Memory for the new Data
36      SinglyList<T>* newNode = new SinglyList<T>;
37      // 2. Put our Data into the New Node
38      newNode->data = newData;
39      // 3. Set Next of the new Node to point to the previous Head
40      newNode->next = *currentHead;
41      // 4. Reset Head to point to the new Node
42      *currentHead = newNode;
43
44  }
```

Insertion at Any part of the List

```
45
46  //GENERAL INSERT
47  template <typename T>
48  void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
49      // 2. If null, print "Previous node cannot be null.
50      if (prevNode == nullptr){
51          std::cout << "Previous Node cannot be Null" << std::endl;
52
53      }
54      // 3. Allocate a new node
55      SinglyList<T>* newNode = new SinglyList<T>;
56      // 4. Store data in the new node
57      newNode->data = newData;
58      // 5. Point new node to the node previous node was pointing to
59      newNode->next = prevNode->next;
60      // 6. Point previous node to the new node
61      prevNode->next = newNode;
62  }
```

Insertion at the End

```cpp
//INSERT END
template <typename T>
void sllInsertEnd(T newData, SinglyList<T>** head){
    //1. Allocate new node
    SinglyList<T>* newNode = new SinglyList<T>;
    //2. Dereference to the head node
    SinglyList<T>* currentNode = *head;
    //3. Store data in new node
    newNode->data = newData;
    //4. Point next of new node to nullptr
    newNode->next = nullptr;
    //5. Traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;

    }
    //6. Point the next of the current node to the new node
    currentNode->next = newNode;

}
```

Deletion of a Node

```cpp

//DELETING NODE
template <typename T>
void sllDeleteNodes(T findData, SinglyList<T>** head){
    // Check if there is no list to be deleted.
    if(*head == nullptr) return;

    SinglyList<T>* currentNode = *head;
    SinglyList<T>* prevNode = nullptr;

    // Searching Loop
    while(currentNode != nullptr && currentNode->data != findData){
        prevNode = currentNode;
        currentNode = currentNode->next;
    }

    // Data was wa NOT found:
    if (currentNode == nullptr) return;

    //What if the Node to be deleted is the Head Node
    if(prevNode == nullptr){
        *head = currentNode->next;
    }else{
        prevNode->next = currentNode->next;
    }

    // We have to DELETE the Node
    delete currentNode;


}
```

Deletion of List	

```
116  //DELETE ALL NODES
117  template <typename T>
118  void sllDeleteList(SinglyList<T>** head){
119      SinglyList<T>* current = *head;
120
121      while(current->next != nullptr){
122          SinglyList<T>* temp = current;
123          current = current->next;
124          delete temp;
125      }
126      *head = nullptr;
127  }
128
129
130
131  #endif
```

Table 3-2. Code for the List Operations

a.	**Source Code**	**Header:**

```
13  //TRAVERSAL
14  template <typename T>
15  void listTraversal(SinglyList<T>* head){
16
17      if (head == nullptr){
18          std::cout << "List is EMPTY" << std::endl;
19          return;
20      }
21
22      while(head != nullptr){
23          std::cout << head->data;
24          if (head != nullptr){
25              std::cout << "->";
26          }
27          head = head->next;
28      }
29      std::cout << std::endl;
30  }
```

Main:

```
32      std::cout << "================================" << std::endl;
33
34      listTraversal(head);
35
36      std::cout << "================================" << std::endl;
```

Console

```
==================================
N->I->N->Y->A->
==================================
```

b.

Source Code

Header:

```
31
32  //INSERT HEAD
33  template <typename T>
34  void sllInsertHead(T newData, SinglyList<T>** currentHead){
35      // 1. Allocate Memory for the new Data
36      SinglyList<T>* newNode = new SinglyList<T>;
37      // 2. Put our Data into the New Node
38      newNode->data = newData;
39      // 3. Set Next of the new Node to point to the previous Head
40      newNode->next = *currentHead;
41      // 4. Reset Head to point to the new Node
42      *currentHead = newNode;
43
44  }
```

Main:

```
36      std::cout << "================================" << std::endl;
37
38      std::cout << "Inserting a New Node at the Head: \n" << std::endl;
39      sllInsertHead('S', &head);
40      listTraversal(head);
41
42      std::cout << "================================" << std::endl;
```

Console

```
==================================
Inserting a New Node at the Head:

S->N->I->N->Y->A->
==================================
```

c.

Source Code

Header:

```
45
46  //GENERAL INSERT
47  template <typename T>
48  void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
49      // 2. If null, print "Previous node cannot be null.
50      if (prevNode == nullptr){
51          std::cout << "Previous Node cannot be Null" << std::endl;
52
53      }
54      // 3. Allocate a new node
55      SinglyList<T>* newNode = new SinglyList<T>;
56      // 4. Store data in the new node
57      newNode->data = newData;
58      // 5. Point new node to the node previous node was pointing to
59      newNode->next = prevNode->next;
60      // 6. Point previous node to the new node
61      prevNode->next = newNode;
62  }
```

Main:

```
42      std::cout << "================================" << std::endl;
43
44      std::cout << "Inserting a New Node at the third: \n" << std::endl;
45      sllGeneralInsert('U', head->next->next);
46      listTraversal(head);
47
48      std::cout << "================================" << std::endl;
```

Console

```
==================================
Inserting a New Node at the third:

S->N->I->U->N->Y->A->
==================================
```

d. **Source Code**

Header:

```
64  //INSERT END
65  template <typename T>
66  void sllInsertEnd(T newData, SinglyList<T>** head){
67      //1. Allocate new node
68      SinglyList<T>* newNode = new SinglyList<T>;
69      //2. Dereference to the head node
70      SinglyList<T>* currentNode = *head;
71      //3. Store data in new node
72      newNode->data = newData;
73      //4. Point next of new node to nullptr
74      newNode->next = nullptr;
75      //5. Traverse the list until next of the node is null
76      while(currentNode->next != nullptr){
77          currentNode = currentNode->next;
78
79      }
80      //6. Point the next of the current node to the new node
81      currentNode->next = newNode;
82
83  }
```

Main:

```
48      std::cout << "================================" << std::endl;
49
50      std::cout << "Inserting a New Node at the End: \n" << std::endl;
51      sllInsertEnd('H', &head);
52      listTraversal(head);
53
54      std::cout << "================================" << std::endl;
```

Console

```
================================
Inserting a New Node at the End:

S->N->I->U->N->Y->A->H->
================================
```

e.

Source Code

Header:

```cpp

//DELETING NODE
template <typename T>
void sllDeleteNodes(T findData, SinglyList<T>** head){
    // Check if there is no list to be deleted.
    if(*head == nullptr) return;

    SinglyList<T>* currentNode = *head;
    SinglyList<T>* prevNode = nullptr;

    // Searching Loop
    while(currentNode != nullptr && currentNode->data != findData){
        prevNode = currentNode;
        currentNode = currentNode->next;
    }

    // Data was wa NOT found:
    if (currentNode == nullptr) return;

    //What if the Node to be deleted is the Head Node
    if(prevNode == nullptr){
        *head = currentNode->next;
    }else{
        prevNode->next = currentNode->next;
    }

    // We have to DELETE the Node
    delete currentNode;


}
```

Main:

```cpp
    std::cout << "================================" << std::endl;

    std::cout << "Deleting a Node: \n" << std::endl;
    sllDeleteNodes('U', &head);
    listTraversal(head);

    std::cout << "================================" << std::endl;

    std::cout << "Deleting the Head Node: \n" << std::endl;
    sllDeleteNodes('S', &head);
    listTraversal(head);

    std::cout << "================================" << std::endl;
```

Console

```
================================
Deleting a Node:

S->N->I->N->Y->A->H->
================================
Deleting the Head Node:

N->I->N->Y->A->H->
================================
```

f.	**Source Code**	**Header:**
		`//DELETE ALL NODES`
		`template <typename T>`
		`void sllDeleteList(SinglyList<T>** head){`
		`  SinglyList<T>* current = *head;`
		
		`  while(current->next != nullptr){`
		`  SinglyList<T>* temp = current;`
		`  current = current->next;`
		`  delete temp;`
		`  }`
		`  *head = nullptr;`
		`}`
		
		`#endif`
		Main:
		`std::cout << "================================" << std::endl;`
		
		`std::cout << "Deleting all the Nodes: \n" << std::endl;`
		`sllDeleteList(&head);`
		`listTraversal(head);`
		
		`std::cout << "================================" << std::endl;`

	Console	`================================`
		`Deleting all the Nodes:`
		
		`List is EMPTY`
		`================================`

Table 3-3. Code and Analysis for Singly Linked List

Screenshot(s)	Analysis

double_LL.h > dllGeneralInsert<T>(T, DoubleList<T>*, DoubleList<T>*)
`#ifndef DOUBLE_LL_H`
`#define DOUBLE_LL_H`
 	
`//TEMPLATING`
`template <typename T>`
`class DoubleList{`
`public:`
`T data;`
`DoubleList<T> *next = nullptr;`
`DoubleList<T> *prev = nullptr;`
 	
`};`
 	
`// Function to Create a new node`
`template <typename T>`
`DoubleList<T>* createNode(T newData) {`
`// Dynamically allocate memory for the new node`
`DoubleList<T>* newNode = new DoubleList<T>;`
 	
`newNode->data = newData;`
`newNode->next = nullptr;`
`newNode->prev = nullptr;`
 	
`// Return the address of the new node`
`return newNode;`
`}`
	This shows the definition of the DoubleList<T> class and the createNode function. This class sets up the basic structure of a doubly linked list node with data, next, and prev pointers, allowing movement in both directions. The createNode function simplifies node creation by allocating memory, initializing values, and returning the new node, which makes insertion functions cleaner and avoids repeated code.

`template <typename T>`
`void dllTraverse(DoubleList<T> *currentNode) {`
 	
`DoubleList<T> *tail = nullptr;`
 	
`// Checking if the list is empty`
`if (currentNode == nullptr) {`
`std::cout << "List is EMPTY" << std::endl;`
`}`
 	
`// Forward Traversal`
`std::cout << "Forward Traversal: " << std::endl;`
`// Moves toward the last node ( before the null )`
`while (currentNode != nullptr) {`
`// Printed the data`
`std::cout << currentNode->data << " ";`
`// Saved the current node as the last visited node`
`tail = currentNode;  // Keep track of the last node for backward traversal`
`// Moves to the next node in the list`
`currentNode = currentNode->next;`
`}`
 	
`//  Backward Traversal`
`std::cout << std::endl << "Backward Traversal: " << std::endl;`
`// Moves toward the head`
`while (tail != nullptr) {`
`// Printed the data`
`std::cout << tail->data << " ";`
`// Move to the previous node in the list`
`tail = tail->prev;`
`}`
`std::cout << std::endl;`
 	
`}`
	This demonstrates the dllTraverse function, which highlights the strength of doubly linked lists: two directional traversal. It first prints the list forward from the head to the tail, then uses the saved tail pointer to print backward toward the head. This shows how the prev pointer makes reverse traversal possible without restarting from the head, something not possible in a singly linked list.

```cpp
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentHead) {
    // Create a new node with the given data
    DoubleList<T>* newNode = createNode(newData);
    // If the list is empty, make the new node the head
    if (*currentHead == nullptr) {
        *currentHead = newNode;
        return;
    }
    // Link the new node to the current head
    newNode->next = *currentHead;
    (*currentHead)->prev = newNode;

    // Update the head to point to the new node
    *currentHead = newNode;
}
```

This focuses on dllInsertHead, which inserts a new node at the beginning of the list. If the list is empty, the new node becomes the head; otherwise, it links the new node to the old head and updates the head pointer. This operation is efficient because it doesn't require traversal, and it demonstrates how the prev pointer ensures proper backward linkage.

```cpp
// Insertion at the end of the list
template <typename T>
void dllInsertTail(T newData, DoubleList<T> *currentHead){
//create
DoubleList<T>* newNode = createNode(newData);
//traverse
    while (currentHead->next != nullptr){
        currentHead = currentHead->next;
    }
    //connect last -> new
    currentHead->next = newNode;
    //connect new -> last
    newNode->prev = currentHead;;
    std::cout << std::endl;
}
```

This shows dllInsertTail, which adds a new node at the end of the list. It traverses until the last node, then connects the new node as the new tail by updating both next and prev pointers. This operation takes linear time since it requires traversal, but it's straightforward and ensures the list remains properly linked in both directions.

```cpp
// Insertion at any position/node
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode, DoubleList<T>* nextNode){
    // Allocate/Create a newNode
    DoubleList<T>* newNode = createNode(newData);
    // Checks if the previous Node is Null, If yes, Print "Previous Node Cannot be NULL"
    if (prevNode == nullptr) {
        std::cout << "Previous Node cannot be NULL" << std::endl;
        delete newNode;
        return;
    }
    // Point the new node to the node that the previous node was pointing to
    newNode->next = nextNode;
    // Set the previous pointer of the new node to the previous node
    newNode->prev = prevNode;
    // Point the previous node to the new node
    prevNode->next = newNode;
    if (nextNode != nullptr) {
        nextNode->prev = newNode; // Update the previous pointer of the next node
    }
}
```

This last part presents dllGeneralInsert, which allows insertion between two existing nodes. It checks for invalid cases like a null prevNode, then carefully updates both next and prev pointers to place the new node in the correct position. This function is the most flexible, as it enables insertion at any point in the list, showing how doubly linked lists provide more control over dynamic data structures.

```
118     // Deletion of a node
119     template <typename T>
120     void dllDeleteNodes(T findData, DoubleList<T>** currentHead) {
121         // If the list is empty, return
122         if (*currentHead == nullptr) {
123             std::cout << "List is EMPTY" << std::endl;
124             return;
125         }
126 
127         DoubleList<T>* currentNode = *currentHead;
128 
129         // Traverse the list to find the node to delete
130         while (currentNode != nullptr && currentNode->data != findData) {
131             currentNode = currentNode->next;
132         }
133 
134         // If the node was not found, return
135         if (currentNode == nullptr) {
136             std::cout << "Node with data " << findData << " not found." << std::endl;
137             return;
138         }
139 
140         if (currentNode == *currentHead) {
141             *currentHead = currentNode->next;
142             if (*currentHead) (*currentHead)->prev = nullptr;
143         } else {
144             currentNode->prev->next = currentNode->next;
145             if (currentNode->next) currentNode->next->prev = currentNode->prev;
146         }
147 
148         delete currentNode;
149     }
150 
151 
152 #endif // DOUBLE_LL_H
```

This is the Delete All Nodes function. Which shows how to completely clear a doubly linked list. The function sllDeleteList starts at the head and iterates through each node, deleting them one by one until the list is empty. By using a temporary pointer (temp) to hold the current node before moving to the next, it ensures that memory is properly freed for every node. After the loop finishes, the head pointer is set to nullptr, which means the list no longer has any nodes and is fully cleared. This operation is important because it prevents memory leaks by making sure all dynamically allocated nodes are deleted. It also resets the list to an empty state, allowing the programmer to reuse the same list structure without leftover data.

Table 3-4. Modified Operations for Doubly Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Task

Header File:

ninsplaylist_LL.h > playlistDeleteSong<T>(T, PlaylistNode<T>**)

```cpp
#ifndef PLAYLIST_LL_H
#define PLAYLIST_LL_H

#include <iostream>
#include <string>
using namespace std;

// Playlist Node (Template)
template <typename T>
class PlaylistNode {
public:
    T data;                         // Song title or data
    PlaylistNode<T>* next = nullptr; // Pointer to next node
};

// Node Creation
template <typename T>
PlaylistNode<T>* createNode(T newData) {
    PlaylistNode<T>* newNode = new PlaylistNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// Playlist Creation (Circular)
// Creates a circular linked list from an array of songs
template <typename T>
PlaylistNode<T>* createPlaylist(T songs[], int size) {
    if (songs == nullptr || size <= 0) return nullptr;

    PlaylistNode<T>* head = nullptr;
    for (int i = 0; i < size; i++) {
        if (i == 0) {
            head = createNode(songs[i]);
            head->next = head; // circular link to itself
        } else {
            PlaylistNode<T>* newNode = createNode(songs[i]);
            PlaylistNode<T>* tail = head;
            while (tail->next != head) {
                tail = tail->next;
            }
            tail->next = newNode;
            newNode->next = head;
        }
    }
    return head;
}
```

```cpp

// Traversal
// Prints all songs in the playlist
template <typename T>
void playlistTraverse(PlaylistNode<T>* head) {
    if (head == nullptr) {
        cout << "Playlist is empty!" << endl;
        return;
    }
    PlaylistNode<T>* current = head;
    do {
        cout << current->data;
        current = current->next;
        if (current != head) cout << " -> ";
    } while (current != head);
    cout << endl;
}

// Play All Songs
template <typename T>
void playlistPlayAll(PlaylistNode<T>* head) {
    cout << "Playing playlist:" << endl;
    playlistTraverse(head);
}

// Insert at Head
template <typename T>
void playlistInsertHead(T newData, PlaylistNode<T>** head) {
    PlaylistNode<T>* newNode = createNode(newData);
    if (*head == nullptr) {
        *head = newNode;
        newNode->next = newNode;
        return;
    }
    PlaylistNode<T>* tail = *head;
    while (tail->next != *head) {
        tail = tail->next;
    }
    newNode->next = *head;
    tail->next = newNode;
    *head = newNode;
}
```

```cpp

// Insert at End
template <typename T>
void playlistInsertEnd(T newData, PlaylistNode<T>** head) {
    PlaylistNode<T>* newNode = createNode(newData);
    if (*head == nullptr) {
        *head = newNode;
        newNode->next = newNode;
        return;
    }
    PlaylistNode<T>* tail = *head;
    while (tail->next != *head) {
        tail = tail->next;
    }
    tail->next = newNode;
    newNode->next = *head;
}

// Insert After a Given Node
template <typename T>
void playlistInsertAfter(T newData, PlaylistNode<T>* prevNode) {
    if (prevNode == nullptr) {
        cout << "Previous song node cannot be null!" << endl;
        return;
    }
    PlaylistNode<T>* newNode = createNode(newData);
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}

// Next Song
template <typename T>
void playlistNext(PlaylistNode<T>*& current) {
    if (current == nullptr) return;
    current = current->next;
}
```

```
126
127 // Previous Song
128 // Finds the node before the current one
129 template <typename T>
130 void playlistPrevious(PlaylistNode<T>* head, PlaylistNode<T>*& current) {
131     if (head == nullptr || current == nullptr) return;
132     PlaylistNode<T>* prev = head;
133     while (prev->next != current && prev->next != head) {
134         prev = prev->next;
135     }
136     if (prev->next == current) current = prev;
137 }
138
139 // Delete a Song by Title
140 template <typename T>
141 void playlistDeleteSong(T findData, PlaylistNode<T>** head) {
142     if (*head == nullptr) return;
143
144     PlaylistNode<T>* current = *head;
145     PlaylistNode<T>* prev = nullptr;
146     bool found = false;
147
148     // Search for the song
149     do {
150         if (current->data == findData) {
151             found = true;
152             break;
153         }
154         prev = current;
155         current = current->next;
156     } while (current != *head);
157
158     if (!found) return;
159
160     // Case 1: Only one node
161     if (current->next == current) {
162         delete current;
163         *head = nullptr;
164         return;
165     }
166
```

```cpp
    // Case 2: Deleting head in multi-node list
    if (prev == nullptr) {
        PlaylistNode<T>* tail = *head;
        while (tail->next != *head) {
            tail = tail->next;
        }
        tail->next = current->next;
        *head = current->next;
        delete current;
        return;
    }

    // Case 3: Deleting non-head node
    prev->next = current->next;
    if (current == *head) *head = current->next;
    delete current;
}

// Delete Entire Playlist
template <typename T>
void playlistDeleteList(PlaylistNode<T>** head) {
    if (*head == nullptr) return;

    PlaylistNode<T>* current = (*head)->next;
    while (current != *head) {
        PlaylistNode<T>* temp = current;
        current = current->next;
        delete temp;
    }
    delete *head;
    *head = nullptr;
}

#endif

```

Main.cpp:

main.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "ninsplaylist_LL.h"   // include the header file with playlist functions

using namespace std;

int main() {
    PlaylistNode<string>* playlist = nullptr;   // start with an empty playlist
    PlaylistNode<string>* current = nullptr;    // pointer to track the current song
    int choice;

    do {
        cout << "\n===== PLAYLIST MENU =====" << endl;
        cout << "1. Create playlist (sample songs)" << endl;
        cout << "2. Insert song at head" << endl;
        cout << "3. Insert song at end" << endl;
        cout << "4. Insert song after current" << endl;
        cout << "5. Play all songs" << endl;
        cout << "6. Show current song" << endl;
        cout << "7. Next song" << endl;
        cout << "8. Previous song" << endl;
        cout << "9. Delete a song" << endl;
        cout << "10. Delete entire playlist" << endl;
        cout << "0. Exit" << endl;
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore(); // clear newline from input buffer

        switch (choice) {
            case 1: {
                // Create a Initial playlist
                string songs[] = {"Intertwined - The Ridleys",
                    "Fallin All in You - Shawn Mendes", "Lemonade - Jeremy Passion",};
                int count = sizeof(songs) / sizeof(songs[0]);
                playlist = createPlaylist(songs, count);   // <-- function call: createPlaylist
                current = playlist;
                cout << "Initial playlist created." << endl;
                playlistTraverse(playlist);               // <-- function call: playlistTraverse
                break;
            }
```

```
41              case 2: {
42                  // Insert at head
43                  string song;
44                  cout << "Enter song title: ";
45                  getline(cin, song);
46                  playlistInsertHead(song, &playlist);       // <-- function call: playlistInsertHead
47                  cout << "Inserted at head." << endl;
48                  playlistTraverse(playlist);                // <-- function call: playlistTraverse
49                  break;
50              }
51              case 3: {
52                  // Insert at end
53                  string song;
54                  cout << "Enter song title: ";
55                  getline(cin, song);
56                  playlistInsertEnd(song, &playlist);        // <-- function call: playlistInsertEnd
57                  cout << "Inserted at end." << endl;
58                  playlistTraverse(playlist);                // <-- function call: playlistTraverse
59                  break;
60              }
61              case 4: {
62                  // Insert after current song
63                  if (current == nullptr) {
64                      cout << "No current song selected!" << endl;
65                      break;
66                  }
67                  string song;
68                  cout << "Enter song title: ";
69                  getline(cin, song);
70                  playlistInsertAfter(song, current);        // <-- function call: playlistInsertAfter
71                  cout << "Inserted after current song." << endl;
72                  playlistTraverse(playlist);                // <-- function call: playlistTraverse
73                  break;
74              }
75              case 5: {
76                  // Play all songs
77                  playlistPlayAll(playlist);                 // <-- function call: playlistPlayAll
78                  break;
79              }
```

```
80              case 6: {
81                  // Show current song
82                  if (current == nullptr) cout << "No current song!" << endl;
83                  else cout << "Current song: " << current->data << endl;
84                  break;
85              }
86              case 7: {
87                  // Move to next song
88                  playlistNext(current);                     // <-- function call: playlistNext
89                  if (current != nullptr) cout << "Now playing: " << current->data << endl;
90                  break;
91              }
92              case 8: {
93                  // Move to previous song
94                  playlistPrevious(playlist, current);       // <-- function call: playlistPrevious
95                  if (current != nullptr) cout << "Now playing: " << current->data << endl;
96                  break;
97              }
```

```
98                case 9: {
99                    // Delete a song
100                   string song;
101                   cout << "Enter song title to delete: ";
102                   getline(cin, song);
103                   playlistDeleteSong(song, &playlist);       // <-- function call: playlistDeleteSong
104                   playlistTraverse(playlist);                // <-- function call: playlistTraverse
105                   break;
106               }
107               case 10: {
108                   // Delete entire playlist
109                   playlistDeleteList(&playlist);             // <-- function call: playlistDeleteList
110                   current = nullptr;
111                   cout << "Playlist deleted." << endl;
112                   playlistTraverse(playlist);                // <-- function call: playlistTraverse
113                   break;
114               }
115               case 0:
116                   cout << "Exiting program..." << endl;
117                   break;
118               default:
119                   cout << "Invalid choice. Try again." << endl;
120           }
121       } while (choice != 0);
122
123       return 0;
124   }
125
```

Output in Console:

```
===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 1
Initial playlist created.
Intertwined - The Ridleys -> Fallin All in You - Shawn Mendes -> Lemonade - Jeremy Passion

===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 2
Enter song title: Take All the Love - Arthur Nery
Inserted at head.
Take All the Love - Arthur Nery -> Intertwined - The Ridleys -> Fallin All in You - Shawn Mendes -> Lemonade - Jeremy Passion

===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 3
Enter song title: I.F.L.Y - Bazzi
Inserted at end.
Take All the Love - Arthur Nery -> Intertwined - The Ridleys -> Fallin All in You - Shawn Mendes -> Lemonade - Jeremy Passion ->
I.F.L.Y - Bazzi
```

```
===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 4
Enter song title: A Very Special Love - Sarah Geronimo
Inserted after current song.
Take All the Love - Arthur Nery -> Intertwined - The Ridleys -> A Very Special Love - Sarah Geronimo -> Fallin All in You - Shawn
 Mendes -> Lemonade - Jeremy Passion -> I.F.L.Y - Bazzi

===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 5
Playing playlist:
Take All the Love - Arthur Nery -> Intertwined - The Ridleys -> A Very Special Love - Sarah Geronimo -> Fallin All in You - Shawn
 Mendes -> Lemonade - Jeremy Passion -> I.F.L.Y - Bazzi

===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 6
Current song: Intertwined - The Ridleys
```

```
===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 7
Now playing: A Very Special Love - Sarah Geronimo
```

```
===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 8
Now playing: Intertwined - The Ridleys

===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 9
Enter song title to delete: I.F.L.Y - Bazzi
Take All the Love - Arthur Nery -> Intertwined - The Ridleys -> A Very Special Love - Sarah Geronimo -> Fallin All in You - Shawn
 Mendes -> Lemonade - Jeremy Passion

===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 10
Playlist deleted.
Playlist is empty!
```

```
===== PLAYLIST MENU =====
1. Create playlist (sample songs)
2. Insert song at head
3. Insert song at end
4. Insert song after current
5. Play all songs
6. Show current song
7. Next song
8. Previous song
9. Delete a song
10. Delete entire playlist
0. Exit
Enter choice: 0
Exiting program...
```

Reflections

Supplementary Code Explanation/Analysis:

The playlist program uses a circular linked list as its foundation, but it borrows ideas from both singly and doubly linked lists. In a circular list, the last node points back to the first node, which makes looping through songs seamless. This is similar to a singly linked list because each node has a next pointer, but it goes further by connecting the end back to the head. That circular connection is what allows the playlist to repeat endlessly without extra logic. At the same time, the program incorporates concepts from a doubly linked list. In a doubly linked list, each node has both a next and a prev pointer, allowing traversal in both directions. While the playlist code primarily uses next pointers, the functions for moving to the previous song mimic the behavior of a doubly linked list by searching for the node before the current one. This shows how the playlist combines the simplicity of singly linked lists with the flexibility of doubly linked lists.

The code provides functions to **create a playlist** from an array of songs, **traverse the playlist** to display all songs, and **play all songs** in sequence. It also includes multiple insertion options: adding a song at the head, at the end, or after a specific / currently playing song. Navigation is supported with functions to move to the **next** or **previous** song, simulating real music player controls just like controlling through your own playlist in youtube. For removal, the program can delete a single song by title or clear the entire playlist, ensuring proper memory management and deletion. The program also has an exit option for a complete experience of interactiveness with the playlist system.

Overall, this supplementary introduces how linked lists can be manipulated and be used as a foundation/reference for data structures and algorithms. Such activities like this playlist can be very challenging but also fire up one's creativity and stock knowledge of other basics. Just like the use of switch cases in this system.

Conclusion:

I can say that this laboratory really helped me understand how linked lists can be applied beyond just theory. At first, I only saw singly and doubly linked lists as abstract data structures, but by building a playlist program, I realized how they can be customized for real-world use. The circular linked list made sense because it allowed the playlist to loop back to the first song, just like how music players repeat tracks. At the same time, the idea of a doubly linked list connected to this because it showed how navigation can go both forward and backward, which is exactly what we do when skipping songs. These operations gave me a clearer picture of how linked lists work in practice. Overall, this lab taught me that linked lists are not just about pointers and nodes, they can be adapted into dynamic systems that can be used in everyday life, like creating a playlist. Moreover, It showed me how circular and doubly linked lists complement each other to make data structures more flexible, it really is getting interesting every step of the way.

Feedback:

This laboratory was honestly challenging, but it was also very rewarding. At first, working with linked lists felt overwhelming because of all the pointer manipulations and the different cases to handle when inserting or deleting nodes. It was easy to make mistakes that caused errors or infinite loops, so I had to be very careful and patient while debugging. But, with the help of our instructor's "Hands-On" teaching I am able to digest this lesson easily as if im learning the basics of C++ when i was a first year student. I would say, the hardest part was the supplementary, I had to understand how circular and doubly linked lists connect to each other and how to properly implement its navigation functions like moving to the next or previous song. Really time consuming and brain eating challenge. I kind of enjoyed it though. THANK YOU SIR TALAGA PROMISE.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.